CONFMAN – A CONFERENCE MANAGEMENT SYSTEM

A MINI PROJECT REPORT

Submitted by

[STUDENT NAME 1] – [REG. NUMBER 1]
[STUDENT NAME 2] – [REG. NUMBER 2]
[STUDENT NAME 3] – [REG. NUMBER 3]

In partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY

LOYOLA-ICAM COLLEGE OF ENGINEERING AND TECHNOLOGY

ANNA UNIVERSITY: CHENNAI 600 025

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that the Mini Project report on **"CONFMAN – A CONFERENCE MANAGEMENT SYSTEM"** is the bonafide work of **[STUDENT NAME 1] ([REG. NUMBER 1]), [STUDENT NAME 2] ([REG. NUMBER 2]), [STUDENT NAME 3] ([REG. NUMBER 3])** who carried out the project work under my supervision.

SIGNATURE

[COORDINATOR NAME],
B.E., M.E.
Project Coordinator
Department of Information Technology
Loyola-ICAM College of Engineering and Technology
Nungambakkam, Chennai-34

SIGNATURE

Ms. Sherril Sophie Maria Vincent
B.E., M.E. , Ph.D.
Head of the Department
Department of Information Technology
Loyola-ICAM College of Engineering and Technology
Nungambakkam, Chennai-34

ACKNOWLEDGEMENT

First of all, we thank The Lord Almighty for showering His blessings upon us, enabling us to complete this project successfully.

We express our sincere gratitude to our beloved and respected Director, **Rev. Dr. Justine SJ**. We wish to acknowledge with thanks to our college Dean of Students, **Rev. Dr. Albert William SJ**, for his support and encouragement throughout this endeavour.

We extend our sincere gratitude to **Dr. A. Baskar**, Principal, for his motivation and support towards this project.

We express our sincere gratitude to the Head of the Department of Information Technology, **Ms. Sherril Sophie Maria Vincent**, for her constant support, motivation, encouragement and for being a source of inspiration throughout this project and during the course of this semester.

Our sincere thanks to our Project Coordinator **[COORDINATOR NAME]** for her valuable guidance in shaping this project.

We also take this opportunity to thank all the Faculty and Non-teaching staff members of the Department of Information Technology for their constant support. Finally, we thank each and every one who helped us to complete this project.

[STUDENT NAME 1]
[STUDENT NAME 2]
[STUDENT NAME 3]

ABSTRACT

The administration of academic peer-reviewed conferences requires coordinated execution across multiple interdependent operational phases: manuscript ingestion, conflict-free reviewer assignment, double-blind peer evaluation, editorial decision-making, fee collection and session scheduling. Legacy conference management systems frequently suffer from usability friction, convoluted interfaces, monolithic architectures and excessive runtime overhead that complicates localised deployment in institutional environments.

This project presents ConfMan, an end-to-end, lightweight, yet functionally comprehensive Conference Management System implemented using Python 3.10+, Flask 3.0+ and SQLite3. The system is engineered around a modular three-tier architecture that decouples web presentation and routing from core domain rules and relational data access. ConfMan implements fine-grained Role-Based Access Control across five distinct user personas: Administrator, Program Chair, Reviewer, Author and Attendee.

Core technical capabilities include an automated reviewer assignment engine driven by keyword and track expertise matching with reviewer workload balancing; an automated editorial decision support system based on score thresholds; strict enforcement of double-blind review constraints and conflict-of-interest prevention; a conference registration engine with dynamic early-bird discount calculation; a payment processing workflow with Luhn algorithm validation and mock gateway interaction; and a conference scheduling module with automated room clash and timeslot collision detection.

The system is validated through an extensive test suite comprising 67 unit and integration tests executing across two test modules, achieving a complete pass rate and verifying business invariants including state transitions, authorisation boundaries and numerical calculations. The repository incorporates a realistic historical demo data generator simulating 3 conferences, 25 users, 30 scholarly papers and 14 payments over a 75-day chronological continuum. The resulting system demonstrates that enterprise-grade scholarly conference management workflows can be realised with zero external runtime dependencies beyond the Flask framework.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iv
	LIST OF FIGURES	vii
	LIST OF TABLES	viii
	LIST OF ABBREVIATIONS	ix
1	INTRODUCTION	1
1.1	Introduction	1
1.2	Problem Statement	1
1.3	Objectives	1
1.4	Scope	2
2	REQUIREMENTS	3
2.1	Functional Requirements	3
2.2	Non-Functional Requirements	4
2.3	Hardware Requirements	4
2.4	Software Requirements	4
3	ANALYSIS	6
3.1	Existing System	6
3.2	Proposed System	6
3.3	Bull Diagram	7
3.4	Octopus Diagram	8
4	SYSTEM DESIGN	9
4.1	System Architecture	9
4.2	Data Flow Diagram	9
4.3	Use Case Diagram	10
4.4	Component Diagram	10
4.5	Sequence Diagram	11
4.6	Activity Diagram	11
4.7	Database Entity Relationship Diagram	12
5	MODULE DESCRIPTION	13
5.1	User and Authentication Module	13

CHAPTER NO.	TITLE	PAGE NO.
5.2	Conference and Track Management Module	13
5.3	Manuscript Ingestion Module	14
5.4	Reviewer Assignment Module	14
5.5	Peer Review and Editorial Decision Module	14
5.6	Registration and Fee Calculation Module	14
5.7	Payment Processing Module	14
5.8	Session Scheduling Module	15
5.9	Analytics and Insights Module	15
6	IMPLEMENTATION	16
6.1	Application Factory and Architecture	16
6.2	Business Logic Layer	16
6.3	Frontend Implementation	17
6.4	Database Schema and Access Patterns	18
6.5	Security Implementation	18
6.6	Deployment	18
7	TESTING AND VERIFICATION	20
7.1	Unit Testing	20
7.2	Functional Testing	21
7.3	Integration Testing	21
7.4	Security Testing	22
8	CONCLUSION AND FUTURE ENHANCEMENTS	23
8.1	Conclusion	23
8.2	Limitations	23
8.3	Future Enhancements	24
9	REFERENCES	25
10	APPENDIX	26
A.	Project Structure	26
B.	Important Commands	26
C.	Demo User Credentials	26
D.	Screenshots	28

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	Bull Diagram	7
3.2	Octopus Diagram	8
4.1	System Architecture	9
4.2	Data Flow Diagram (Level 0)	10
4.3	Use Case Diagram	10
4.4	Component Diagram	11
4.5	Sequence Diagram	11
4.6	Activity Diagram – Paper Lifecycle	12
4.7	Entity Relationship Diagram	12
6.1	Conference Lifecycle Workflow	19
D.1	Dashboard – Home Page and Login Screen	28
D.2	Chair Dashboard – KPI Cards and Submission Queue	28
D.3	Paper Detail View with Review Panel	28
D.4	Payment Checkout and Registration Confirmation	28

LIST OF TABLES

Table No.	Title	Page No.
2.1	Tools and Technologies Used	4
3.1	Comparison of Existing and Proposed Systems	6
5.1	Major Project Modules	13
6.1	HTTP API Endpoints	16
7.1	Automated Test Cases (Representative Sample)	20
7.2	Manual Functional Verification Checklist	21
8.1	Implementation Status Summary	23

LIST OF ABBREVIATIONS

S. No.	Abbreviation	Expansion
1	API	Application Programming Interface
2	ACID	Atomicity, Consistency, Isolation, Durability
3	CFP	Call for Papers
4	COI	Conflict of Interest
5	CSRF	Cross-Site Request Forgery
6	CSS	Cascading Style Sheets
7	CSV	Comma-Separated Values
8	DDL	Data Definition Language
9	FK	Foreign Key
10	HTML	HyperText Markup Language
11	HTTP	HyperText Transfer Protocol
12	INR	Indian National Rupee
13	PAN	Primary Account Number
14	PK	Primary Key
15	RBAC	Role-Based Access Control
16	REST	Representational State Transfer
17	SQL	Structured Query Language
18	SVG	Scalable Vector Graphics
19	UUID	Universally Unique Identifier
20	WSGI	Web Server Gateway Interface

CHAPTER 1: INTRODUCTION

1.1 INTRODUCTION

Academic conferences constitute the primary dissemination channel for rapidly advancing scientific disciplines, particularly in computer science and engineering. Organising a contemporary conference entails managing complex, non-linear workflows: configuring tracks, accepting scholarly submissions, maintaining double-blind peer review integrity, resolving reviewer–author conflicts of interest (COI), orchestrating reviewer bidding and assignment, computing aggregate review scores, rendering editorial decisions, processing tiered registrations with early-bird discounts, verifying payment transactions and compiling accepted papers into multi-session presentation programmes without room or timeslot collisions.

ConfMan is a fully realised, self-contained, multi-role conference management web application engineered to address these operational requirements without the operational weight, licensing friction or hosting complexity of legacy enterprise platforms. The system is implemented using Python 3.10+, Flask 3.0+ and SQLite3, adhering strictly to the separation of concerns principle. ConfMan encapsulates all business logic within a framework-agnostic service layer (cms_app/services.py), leverages analytical read models for real-time dashboards (cms_app/insights.py), provides role-based user interfaces and utilises SQLite3 with ACID transactions and schema-level relational constraints.

The system natively supports five distinct actors with strictly enforced Role-Based Access Control: System Administrator, Program Committee Chair, Reviewer, Author and Conference Attendee. Together, these roles cover the complete academic conference lifecycle from call-for-papers through manuscript review, editorial decision, registration and payment, to final session scheduling and programme publication.

1.2 PROBLEM STATEMENT

Traditional academic conference coordination workflows are characterised by administrative fragmentation, manual communication bottlenecks and inadequate automated validation. The principal challenges include:

- **Workload Imbalance.** Manual reviewer assignment without real-time visibility into workloads leads to reviewer burnout, missed deadlines and low evaluation quality.
- **Conflict of Interest Vulnerabilities.** Manual or spreadsheet-based systems allow authors to be inadvertently assigned to review their own papers or those of institutional colleagues.
- **Double-Blind Integrity Violations.** Author identities and reviewer critiques are frequently exposed prematurely in manual workflows, introducing prestige bias into scholarly evaluation.
- **Complex Registration and Fee Reconciliation.** Calculating early-bird versus standard fees across diverse attendee categories requires cumbersome financial reconciliation and is prone to arithmetic errors.
- **Scheduling and Venue Clashes.** Manually organising accepted papers into rooms and timeslots frequently produces room double-booking, over-capacity allocations or scheduling papers whose authors have not registered.
- **Administrative Overhead.** Existing enterprise systems (EasyChair, EDAS, HotCRP) require dedicated database servers, message brokers and paid subscriptions, creating substantial hurdles for lightweight deployment in university departments.

1.3 OBJECTIVES

The main objectives of the ConfMan project are:

1. To develop a multi-role relational web application supporting users, conferences, tracks, papers, review assignments, reviews, decisions, registrations, payments, sessions and presentations with full foreign-key referential integrity.
2. To enforce Role-Based Access Control using declarative decorators and session verification, preventing unauthorised privilege escalation.
3. To implement both manual reviewer selection and a greedy automated assignment algorithm that ranks candidates based on keyword and track expertise overlap while enforcing capacity limits and COI prevention.
4. To guarantee double-blind review integrity by preventing reviewers from accessing author names and affiliations, and restricting authors from viewing critique details before an official editorial decision is recorded.
5. To provide algorithmic editorial decision guidance by computing composite average scores and suggesting acceptance, rejection or revision based on empirical score thresholds.
6. To implement a tiered registration engine with automatic early-bird discount calculation (20% reduction on or before the early-bird deadline).
7. To simulate secure payment processing with Luhn mod-10 credit card validation, transaction reference generation and zero retention of sensitive card numbers.
8. To enable clash-free session scheduling with room double-booking detection and mandatory author registration verification before paper scheduling.
9. To deliver role-specific analytical dashboards with real-time KPI metrics, SVG sparklines and weekly submission time-series charts.
10. To validate all core business logic through an automated test suite achieving a 100% pass rate across 67 unit and integration test cases.

1.4 SCOPE

The scope of ConfMan encompasses five distinct user interaction models. The System Administrator manages global user lifecycle events, toggles account active states, reassigns conference chairs and inspects platform-wide KPI dashboards and security audit logs. The Program Chair creates and configures conferences with submission and early-bird deadlines, defines thematic tracks, assigns reviewers manually or through the automated ranking heuristic, records editorial decisions and schedules accepted papers into sessions.

The Reviewer browses the pool of manuscripts requiring evaluation, self-assigns through the volunteering mechanism, submits quantitative scores (1–10) and narrative critiques in double-blind mode and tracks pending workload against the maximum quota of five concurrent reviews. The Author submits PDF manuscripts with title, abstract and keywords, monitors real-time submission status, inspects anonymous review feedback after a decision is recorded and registers for the conference. The Attendee discovers conferences, selects a registration tier, completes mock card payment and browses the accepted paper programme.

The system is designed as a single-node WSGI application backed by SQLite3, without external database daemons, message brokers or JavaScript build pipelines. It is intended as an educational prototype and institutional reference implementation, not as a production replacement for large-scale enterprise conference management platforms.

CHAPTER 2: REQUIREMENTS

2.1 FUNCTIONAL REQUIREMENTS

The system shall implement the following functional capabilities:

1. User Registration and Role Selection. The system shall allow public users to register as Author, Reviewer or Attendee. Self-registration as Chair or Admin shall be blocked. Passwords shall be at least 8 characters long and contain at least one numeric digit.
2. User Authentication and Session Security. The system shall authenticate users via email and PBKDF2-HMAC-SHA256 password hash comparison. Deactivated accounts shall be refused login.
3. Conference Creation and Configuration. Program Chairs and Admins shall create conferences with venue, dates, submission deadline and early-bird deadline, with chronological consistency enforced.
4. Track Management. Chairs shall add thematic tracks to their conference with uniqueness of track names enforced within each conference.
5. Manuscript Submission. Authors shall submit papers by selecting a conference and track, providing title (max 200 characters), abstract (min 50 characters), keywords and a PDF document. Submissions shall be refused after the submission deadline.
6. Manual Reviewer Assignment. Chairs shall manually assign active reviewers. COI prevention (author cannot review own paper), duplicate assignment prevention and the cap of 3 reviewers per paper and 5 pending reviews per reviewer shall be enforced.
7. Reviewer Volunteering. Reviewers shall browse open papers and self-assign, subject to identical COI, duplicate and capacity constraints as chair assignment.
8. Automated Reviewer Allocation. A greedy algorithm shall rank candidate reviewers by keyword and track expertise overlap, prioritising lower workload among equally matched candidates, and auto-assign up to 3 reviewers per paper.
9. Double-Blind Review Evaluation. Reviewers shall evaluate assigned submissions anonymously. Submission shall include an integer score (1–10), narrative critique (min 20 characters) and a recommendation.
10. Algorithmic Decision Support. The system shall calculate composite review scores and suggest an editorial decision when at least 2 reviews are complete: Accepted (average >= 7.0), Revision Required (5.0–6.9) or Rejected (< 5.0).
11. Editorial Decision Recording. Chairs shall record binding decisions with optional remarks. Decisions shall require at least 2 completed reviews.
12. Tiered Conference Registration. The system shall support Student (Rs 500), Academic (Rs 1,000) and Industry (Rs 2,000) tiers with an automatic 20% early-bird discount.
13. Payment Processing Simulation. The system shall validate credit card numbers using the Luhn mod-10 algorithm, decline numbers ending in 0002 via the mock gateway and never persist card numbers.
14. Clash-Free Session Scheduling. Chairs shall define sessions with room, capacity and time window. Room double-booking shall be blocked. Only accepted papers whose authors hold a confirmed, paid registration may be scheduled.
15. Administration and Audit Logging. Admins shall manage user accounts, roles and conference assignments, with all significant actions recorded in an immutable activity audit log.

2.2 NON-FUNCTIONAL REQUIREMENTS

Security. All passwords are stored as PBKDF2-HMAC-SHA256 hashes. All POST requests require CSRF token verification using constant-time string comparison. Reviewer and author views strictly enforce double-blind isolation. SQL injection is prevented through 100% parameterised query bindings. Card numbers are validated in memory and immediately discarded.

Reliability. Referential integrity is enforced through SQLite foreign keys enabled via PRAGMA foreign_keys = ON. Strict database check constraints apply to user roles, paper statuses, review scores and payment states. All mutating service routines use explicit transactions with commit and rollback on error.

Performance. Cold-boot startup completes in under 200 milliseconds. All 67 automated tests execute in approximately 31 seconds on standard hardware. Analytical SQL queries in insights.py complete within single-digit milliseconds against the SQLite B-tree indices.

Usability. The interface is fully responsive, adapting from mobile viewports to wide-screen desktops. A built-in dark and light mode toggle persists user preference in browser localStorage. Role-specific dashboards provide SVG sparklines, ring charts and colour-coded status badges.

Maintainability. All business rules reside in cms_app/services.py and take standard SQLite3 database connections, enabling unit testing without Flask HTTP contexts. The three-tier architecture provides distinct presentation, domain service, read model and persistence modules.

Portability. The application runs on Linux, macOS and Windows with Python 3.10–3.14. The sole external runtime dependency is Flask >= 3.0. No external database server, cache or message broker is required.

2.3 HARDWARE REQUIREMENTS

- Development machine: computer or laptop with minimum 4 GB RAM and 100 MB free disk space.
- Active internet connection for pip package installation (initial setup only).
- Users: any computer, tablet or smartphone with a modern web browser (Chrome, Firefox, Edge, Safari).

2.4 SOFTWARE REQUIREMENTS

- Operating System: Linux (Ubuntu 20.04+), macOS 11+ or Windows 10/11.
- Python runtime: Python 3.10, 3.11, 3.12, 3.13 or 3.14.
- Web framework: Flask >= 3.0 (installed via pip).
- Modern web browser for the user interface.
- Git and GitHub for version control.
- Visual Studio Code or equivalent development environment.

Table 2.1 – Tools and Technologies Used

Technology	Role in the Project
Python 3.10+	Core application language; business rules, services and database operations.
Flask 3.0+	WSGI web framework; routing, session management, template rendering and error handling.
Werkzeug 3.0+	Password hashing (PBKDF2:SHA256), secure filename sanitisation, bundled with Flask.
Jinja2 3.1+	Server-side HTML template compilation; custom filters (dt, inr, ago, firstname).

Technology	Role in the Project
SQLite3	Relational transactional data store with ACID semantics and foreign key enforcement.
Vanilla CSS3	Custom Teal Design System with CSS Grid, Flexbox and dark and light theme variables.
Inline SVG	Icons, KPI sparklines, ring charts and bar chart visualisations.
Python unittest	Standard library test harness for unit and integration test execution.
pip	Python package manager; single dependency (Flask >= 3.0).
Git / GitHub	Version control and repository hosting.

CHAPTER 3: ANALYSIS

3.1 EXISTING SYSTEM

Academic event management has historically relied on two models. The first is ad-hoc manual coordination: small workshops and symposia manage submissions, assignments and reviews via email attachments, shared folders such as Google Drive and spreadsheets. The second is legacy enterprise platforms: larger conferences deploy established SaaS or on-premise platforms such as EasyChair, EDAS, OpenConf or HotCRP.

Both approaches suffer from documented limitations:

- Conflict-of-interest prevention is highly error-prone in manual systems. In legacy platforms, enforcement is rigid and often fails to adapt to track-based configurations or self-assignment volunteer models.
- Reviewer workload is difficult to track in real time, leading to reviewer fatigue, missed deadlines and low evaluation quality.
- Manual workflows rely on forwarding raw PDF emails, making double-blind integrity easy to breach inadvertently.
- Legacy platforms carry dense, early-2000s web interfaces with poor mobile responsiveness, high subscription fees and mandatory external database infrastructure (MySQL or PostgreSQL).
- Financial reconciliation across student, academic and industry attendee categories requires cumbersome manual calculations or disjointed third-party payment forms.
- Scheduling accepted papers manually into physical rooms and timeslots frequently produces double-booking, over-capacity allocations or papers by unregistered authors.

Table 3.1 – Comparison of Existing and Proposed Systems

Feature	Manual / Spreadsheet Systems	ConfMan (Proposed System)
COI Prevention	Error-prone; relies on human memory.	Automated; reviewer_id != author_id enforced in service layer.
Reviewer Load Balancing	Difficult to track in real time.	Real-time workload metrics; max 5 pending reviews per reviewer.
Double-Blind Integrity	Easily breached when forwarding emails.	Enforced at template and routing level; author identity stripped.
User Interface	Email threads, spreadsheets and file drives.	Responsive web dashboard with role-specific views and dark mode.
Deployment Complexity	No system needed.	pip install Flask; zero external database or cache services.
Payment Processing	Manual bank transfers or disjointed forms.	Luhn-validated mock payment gateway with tiered fee engine.
Session Scheduling	Manual; prone to room clashes.	Automated clash detection; registration prerequisite enforced.
Analytics	Not available.	Real-time KPI cards, SVG sparklines and score histograms.

3.2 PROPOSED SYSTEM

ConfMan resolves the documented limitations through a streamlined, architecturally decoupled web platform built from first principles. The system introduces a strict three-tier layered design that isolates all business rules in cms_app/services.py, taking raw SQLite3 database connections and raising structured ServiceError exceptions. This ensures 100% testability of business logic independent of HTTP request contexts.

A dedicated read-model engine (cms_app/insights.py) encapsulates complex dashboard metrics, time-series bucketing and analytics pipelines, preserving clean separation between transactional write operations and analytical read operations. The integrated lifecycle automation coordinates all phases — submission, double-blind review, editorial scoring thresholds, category-tiered payment and room-clash-free scheduling — through shared relational invariants enforced at the database layer.

The intended users are academic departments and research institutions seeking a lightweight, open-source reference implementation for conference management. The system is deployable with a standard Python installation and a single pip install command, with zero external database configuration, message queues or asset compilation pipelines required.

3.3 BULL DIAGRAM

The Bull Diagram represents the central problem of manual, fragmented academic conference coordination, together with its root causes and downstream effects.

Central Problem: Manual, fragmented and error-prone academic conference coordination workflows.

Major Causes

- Reviewer assignment performed manually without real-time workload visibility.
- Conflict-of-interest screening reliant on human memory and cross-referencing.
- No automated double-blind enforcement mechanism in spreadsheet-based systems.
- Tiered fee calculation and early-bird discounts computed manually.
- Room and timeslot scheduling done via spreadsheets without clash detection.
- Heavy external infrastructure requirements for legacy enterprise platforms.

Effects

- Reviewer overload, fatigue and missed review deadlines.
- Ethical breaches from inadvertent author self-review assignments.
- Premature author de-anonymisation and evaluation bias.
- Financial reconciliation discrepancies and registration errors.
- Room double-booking and scheduling of unregistered authors.
- High subscription fees and maintenance overhead for legacy systems.

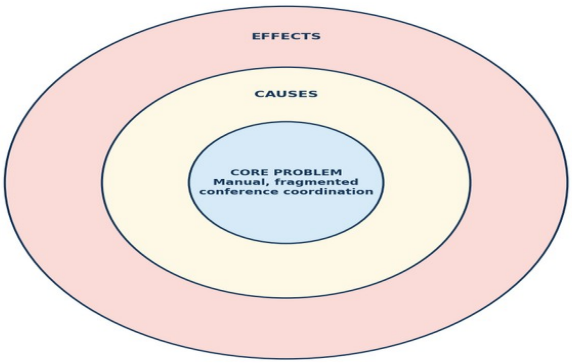


Figure 3.1 – Bull Diagram

3.4 OCTOPUS DIAGRAM

The central system is ConfMan – A Conference Management System. The major components connected to it are:

- User Authentication and Role-Based Access Control
- Conference and Track Management Module
- Manuscript Submission Engine
- Reviewer Assignment Heuristic (Manual, Auto-Assign and Volunteering)
- Double-Blind Peer Review and Editorial Decision Engine
- Registration and Fee Calculation Module
- Payment Processing and Mock Gateway
- Session Scheduling and Programme Module

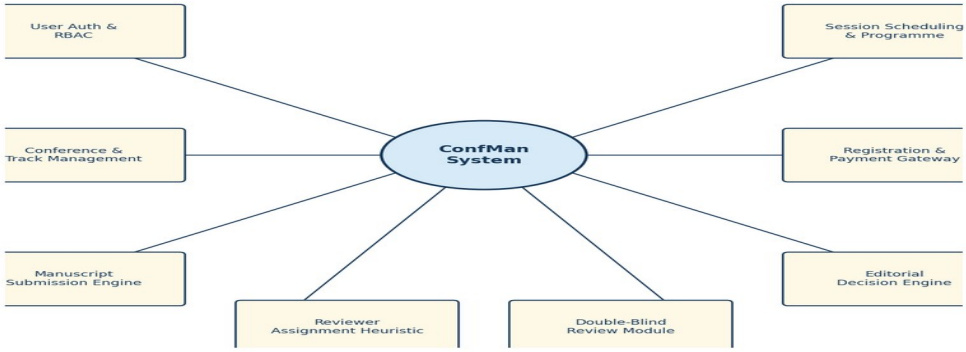


Figure 3.2 – Octopus Diagram

CHAPTER 4: SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

ConfMan is architected as a modular three-tier monolithic application. Unlike sprawling microservice architectures that introduce network latency and distributed transaction complexity, ConfMan adopts in-process architectural separation that delivers high reliability with minimal operational overhead.

The Presentation Tier consists of the Flask WSGI application factory (cms_app/__init__.py), which handles 26 HTTP route controllers, security pre-filters through before_request hooks, CSRF token verification using secrets.compare_digest, and role-based access decorators (@login_required and @role_required). Jinja2 templates and a custom Teal Design System stylesheet render the role-specific user interfaces. The Business Logic Tier houses cms_app/services.py for transactional domain operations and cms_app/insights.py for read-model analytics. The Data Access Tier manages thread-local SQLite3 connections through cms_app/db.py, enforcing PRAGMA foreign_keys = ON and providing the schema initialiser. All manuscript PDF files are stored on the local filesystem at instance/uploads/.

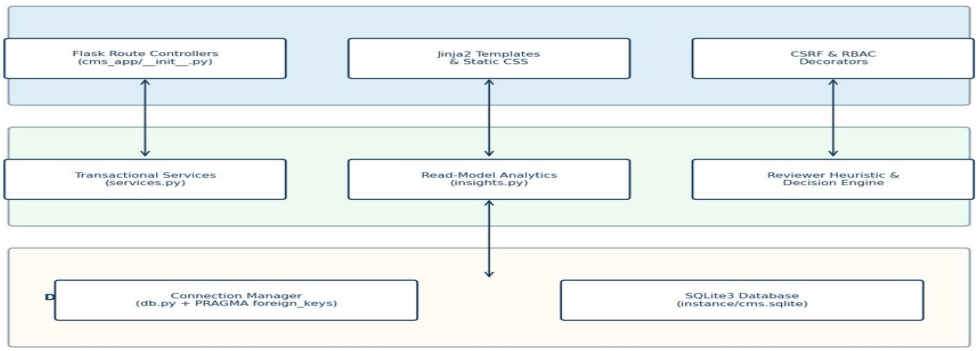


Figure 4.1 – System Architecture

4.2 DATA FLOW DIAGRAM

Level 0 DFD

The user provides a user identity, conference selections and manuscript files to the ConfMan system, and receives role-specific dashboards, review feedback, payment receipts and conference schedules. The system reads from and writes to the SQLite3 database (user records, conference data, papers, assignments, reviews, decisions, registrations, payments, sessions and audit logs) and the local file system (PDF manuscripts). External entities are the five user roles (Administrator, Chair, Reviewer, Author and Attendee). Data stores are D1 – SQLite3 Database (instance/cms.sqlite) and D2 – File Storage (instance/uploads/).

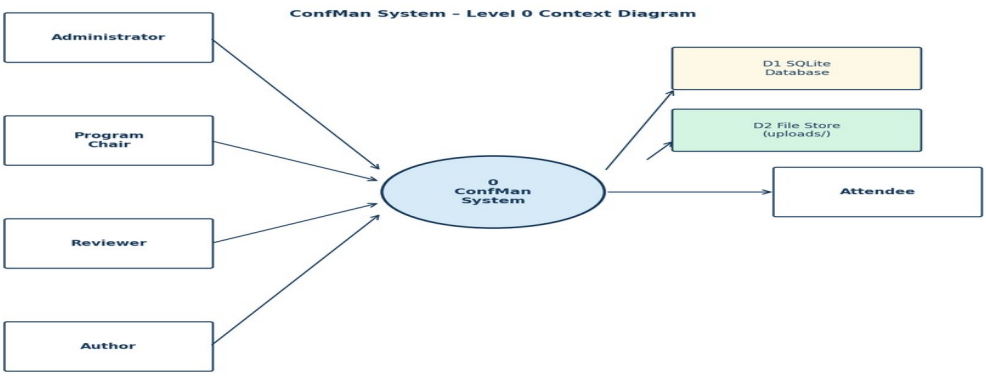


Figure 4.2 – Data Flow Diagram

4.3 USE CASE DIAGRAM

Actors: Administrator, Program Chair, Reviewer, Author and Attendee.

Use Cases

- Register or log in to the system.
- Create conference with tracks and configure scheduling parameters (Chair or Admin).
- Submit a manuscript with PDF upload and metadata (Author).
- Assign or auto-assign reviewers using the expertise heuristic (Chair or Admin).
- Submit a double-blind peer review with score and narrative critique (Reviewer).
- Record an editorial decision after at least 2 reviews are completed (Chair).
- Register for a conference and complete a mock card payment (Author or Attendee).
- Create sessions and schedule accepted papers into presentation slots (Chair).
- Browse the public conference programme and view analytics (Attendee or Chair).

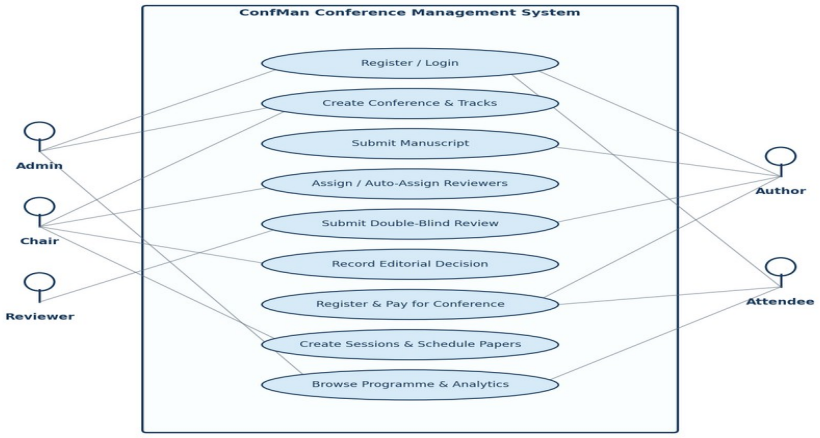


Figure 4.3 – Use Case Diagram

4.4 COMPONENT DIAGRAM

The major components of ConfMan and their connections are illustrated in Figure 4.4. The Web Browser communicates with the Flask Route Controllers over HTTP, which in turn delegate to services.py for transactional operations and insights.py for analytical read models. Both service modules interact with the SQLite3 database through the Connection Manager (db.py) backed by schema.sql. Manuscript PDF files are read and written to the File Storage component. Jinja2 templates and the CSS stylesheet consume data from

the route controllers to render the user interface. The automated test suite (tests/) directly calls the service functions against in-memory SQLite3 instances.

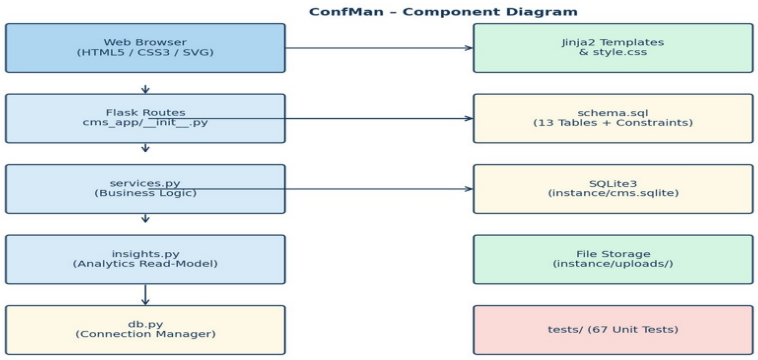


Figure 4.4 – Component Diagram

4.5 SEQUENCE DIAGRAM

The sequence diagram in Figure 4.5 traces the end-to-end manuscript submission workflow. The Author submits a POST request to /conferences/{id}/submit containing the manuscript title, abstract, keywords and PDF file. The Flask Route Controller validates the CSRF token and author role, then delegates to services.submit_paper(). The service validates the submission deadline and track assignment, inserts the paper record with status Submitted, dispatches notifications to both the author and conference chair, commits the transaction and returns the new paper_id. The controller saves the uploaded PDF to instance/uploads/ and redirects the browser to the paper detail page.

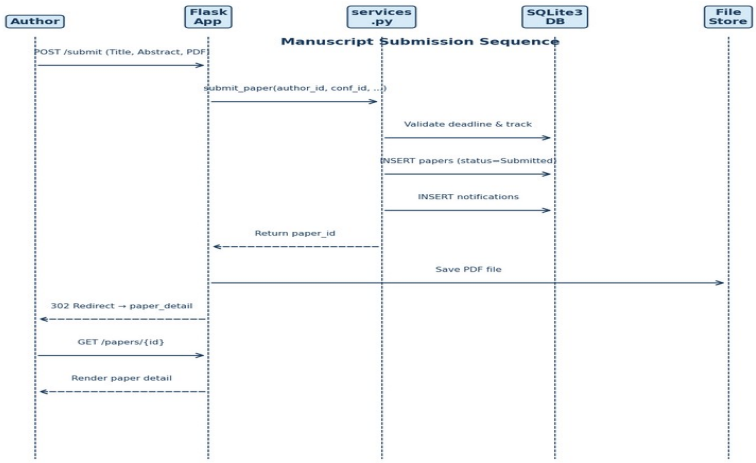


Figure 4.5 – Sequence Diagram

4.6 ACTIVITY DIAGRAM

The activity diagram in Figure 4.6 illustrates the paper lifecycle from initial submission through final session scheduling. There are four key decision points in the flow: whether the submission deadline has elapsed; whether all assigned reviews have been completed; whether the chair records Accepted, Rejected or Revision Required; and whether the author holds a confirmed, paid registration (the mandatory prerequisite for paper scheduling into a session slot).

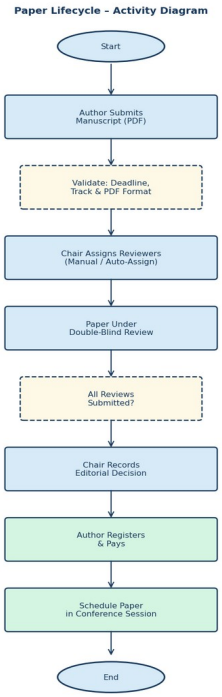


Figure 4.6 – Activity Diagram – Paper Lifecycle

4.7 DATABASE ENTITY RELATIONSHIP DIAGRAM

ConfMan uses a SQLite3 relational database (instance/cms.sqlite) comprising 13 tables with strict column constraints, primary key auto-increment, unique composite indices and domain check constraints. The core entities are users, conferences, tracks, papers, review_assignments, reviews, decisions, registrations, payments, sessions, presentations, notifications and activity. Foreign key cascades are enforced for all referencing relationships. The ERD in Figure 4.7 illustrates the key tables and their relationships.

The users table is the central anchor: a user can chair a conference, author papers, hold review assignments and register for conferences. The papers table links a conference, a track and an author, progressing through statuses of Submitted, Under Review, Reviewed, Accepted or Rejected, and finally Scheduled. Each paper can have up to three review_assignments, each yielding one review record. An accepted paper may have exactly one presentation record linking it to a session.

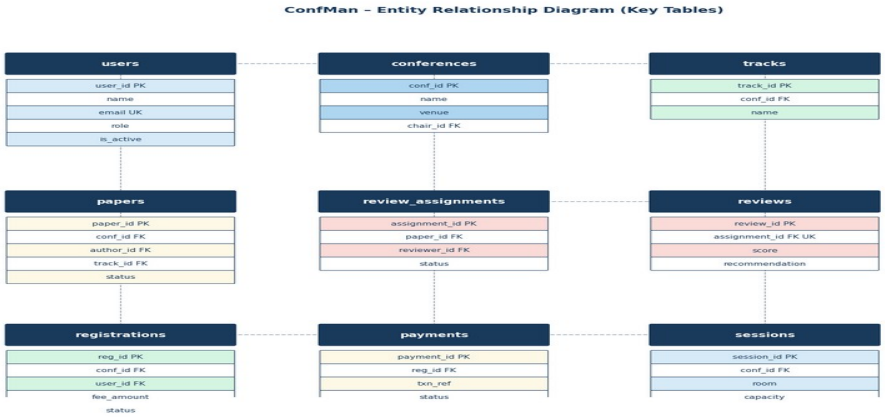


Figure 4.7 – Entity Relationship Diagram

CHAPTER 5: MODULE DESCRIPTION

ConfMan is structured into nine distinct functional modules. Each module has a clearly defined responsibility, and together they provide the complete conference management lifecycle.

Table 5.1 – Major Project Modules

Module	Primary File	Responsibility
User & Authentication	services.py L101–135	User registration, PBKDF2 hashing, login and session management.
Conference & Tracks	services.py L138–182	Conference creation, date validation, track management and CSV export.
Manuscript Ingestion	services.py L185–217	PDF submission, deadline validation, metadata indexing and file storage.
Reviewer Assignment	services.py L220–348	Manual, auto-assign and volunteer allocation with COI and workload checks.
Peer Review & Decision	services.py L350–426	Blind review submission, score aggregation and editorial decision recording.
Registration & Fee	services.py L428–455	Tiered fee engine with early-bird discount and duplicate registration prevention.
Payment Processing	services.py L457–500	Luhn validation, mock gateway, transaction reference and card zero-retention.
Session Scheduling	services.py L502–578	Room clash detection, capacity checks and paid-author prerequisite enforcement.
Analytics & Insights	insights.py L1–219	KPI aggregations, weekly time-series, reviewer workload and score histograms.
Presentation Routing	__init__.py L1–928	Flask routes, CSRF guard, role decorators and Jinja2 context processors.
Tests	tests/	67 automated unit and integration test cases achieving 100% pass rate.

5.1 USER AND AUTHENTICATION MODULE

The User and Authentication Module is the entry point for every interaction. It accepts a user's name, email, password and role, normalises the email to lowercase and validates password complexity (minimum 8 characters, at least one numeric digit). Passwords are stored as Werkzeug PBKDF2-HMAC-SHA256 hashes. Self-registration is restricted to the Author, Reviewer and Attendee roles; Chair and Admin accounts must be created by an existing Administrator. During login, the module verifies the hash and checks the is_active flag, refusing deactivated accounts with a descriptive error.

5.2 CONFERENCE AND TRACK MANAGEMENT MODULE

This module enables Program Chairs and Administrators to create conference editions with venue, start and end dates, submission deadline and early-bird deadline. The service enforces chronological consistency: end_date >= start_date, submission_deadline < start_date and early_bird_deadline <= start_date. Tracks are added as thematic areas within a conference; the schema enforces UNIQUE (conf_id, name) to prevent duplicate track names within the same conference. The module also provides get_program for public timetable rendering and export_papers for RFC 4180 compliant CSV export of paper metadata.

5.3 MANUSCRIPT INGESTION MODULE

The Manuscript Ingestion Module processes author paper submissions. It validates that the submission deadline has not elapsed, that the specified track belongs to the target conference, that the title does not exceed 200 characters, that the abstract contains at least 50 characters and that the uploaded file has a .pdf extension. On success, it inserts a paper record with status Submitted, dispatches in-app notifications to both the submitting author and the conference chair, logs the event in the activity table and commits the transaction. The uploaded PDF is saved to instance/uploads/ using a UUID-based filename pattern to prevent collision and directory traversal attacks.

5.4 REVIEWER ASSIGNMENT MODULE

The Reviewer Assignment Module provides three allocation pathways. The manual pathway allows the chair to select a specific reviewer for a paper. The automated pathway implements a greedy ranked algorithm that tokenises paper keywords, track name and title into term sets, computes keyword overlap with each candidate reviewer's expertise field and ranks candidates by descending overlap, then ascending pending workload. The volunteering pathway allows reviewers to self-assign from a curated open papers pool. All three pathways enforce identical invariants: the reviewer must not be the paper's author (COI prevention), cannot already be assigned to the same paper, cannot hold more than 5 pending reviews and the paper cannot have more than 3 reviewers. Assigning the first reviewer transitions the paper status to Under Review.

5.5 PEER REVIEW AND EDITORIAL DECISION MODULE

Reviewers submit evaluations consisting of an integer score (1–10), a narrative critique (minimum 20 characters) and a categorical recommendation. Author identity is stripped from all reviewer-facing views; reviewer identities and critiques are hidden from authors until a decision is recorded. When all pending review assignments for a paper are completed, the paper status automatically transitions to Reviewed. The decision support function calculates the arithmetic mean of completed review scores and maps it to a suggested decision: Accepted if the average is 7.0 or above, Revision Required if between 5.0 and 6.9 and Rejected if below 5.0. The chair records a binding decision after at least 2 reviews are present, with the system dispatching an author notification containing the decision remarks.

5.6 REGISTRATION AND FEE CALCULATION MODULE

This module manages conference attendee registrations and fee determination. Three registration tiers are supported: Student (Rs 500), Academic (Rs 1,000) and Industry (Rs 2,000). If the registration date falls on or before the conference early_bird_deadline, a 20% discount is applied automatically (fee *= 0.80). Duplicate registrations for the same user and conference are blocked by a UNIQUE (conf_id, user_id) database constraint. Each registration begins in Pending Payment status and transitions to Confirmed after a successful payment.

5.7 PAYMENT PROCESSING MODULE

The Payment Processing Module validates credit card instruments and simulates gateway interaction. Card numbers must consist of 13 to 19 digits and pass the Luhn mod-10 checksum algorithm. The mock gateway declines any card whose digits end with 0002; all other valid cards receive a cryptographically pseudo-random transaction reference in the format TXN-{10 hex chars}. Critically, card numbers are never stored in the database or written to any log file, consistent with PCI-DSS compliance simulation. Only the transaction reference, timestamp, fee amount and outcome status are persisted in the payments table.

5.8 SESSION SCHEDULING MODULE

The Session Scheduling Module enables chairs to define conference presentation sessions with room designators, timeslots and seating capacity. Before creating a session, the module queries existing sessions for the same conference and room to detect temporal overlap using the interval condition: existing.start_time < new_end_time AND existing.end_time > new_start_time. Overlapping sessions are rejected. When scheduling an accepted paper into a session, the module verifies that the paper status is Accepted, that the session has not reached capacity, that the paper has not already been scheduled and that the paper's primary author holds a confirmed, paid registration. On success, a presentations record is inserted and the paper status advances to Scheduled.

5.9 ANALYTICS AND INSIGHTS MODULE

The Analytics and Insights Module (cms_app/insights.py) serves read-only analytical queries separated entirely from transactional write logic. Functions include manager_kpis for computing total submissions, acceptance percentages and review completion rates; admin_kpis for platform-wide user growth and revenue metrics; weekly_series for 8-week submission velocity bucketing used in SVG bar charts; reviewer_workload for pending and completed review counts per reviewer; open_papers for constructing the reviewer volunteering pool; and score_histogram for review score distribution charts. All aggregations execute directly in SQLite3 in single-digit milliseconds.

CHAPTER 6: IMPLEMENTATION

6.1 APPLICATION ARCHITECTURE AND FACTORY PATTERN

The entire application is assembled by the create_app() factory function in cms_app/__init__.py. The factory accepts an optional test_config dictionary, enabling the test harness to substitute an in-memory SQLite database and disable CSRF enforcement. Key configuration parameters include SECRET_KEY (read from the CMS_SECRET environment variable), DATABASE (path to instance/cms.sqlite), UPLOAD_FOLDER (path to instance/uploads/), MAX_CONTENT_LENGTH (10 MB ceiling for PDF uploads) and AUTO_SEED (automatically loads demo data into an empty database at startup). The factory registers the database teardown hook, custom Jinja2 filters and all route handlers before returning the configured application instance.

The security pre-filter attached via @app.before_request intercepts every incoming request. It reads session['user_id'], queries the active user record, attaches it to Flask's context proxy as g.user and, for POST requests, extracts the form field _csrf and compares it to session['_csrf'] using secrets.compare_digest. Mismatches return HTTP 400 immediately. The route-level decorators @login_required and @role_required enforce authentication and authorisation, returning HTTP 302 or HTTP 403 respectively when checks fail.

6.2 BUSINESS LOGIC LAYER

The business logic layer is implemented entirely in cms_app/services.py. Every service function accepts a standard sqlite3.Connection as its first argument and raises a ServiceError exception (a subclass of ValueError) on rule violations. This design ensures that the services have zero imports from Flask, Werkzeug or Jinja2, making them independently testable with in-memory SQLite instances.

The reviewer ranking heuristic is particularly noteworthy. A terms() helper tokenises text by splitting on non-alphanumeric characters and converting tokens to lowercase, discarding terms of 2 characters or fewer. The wanted set is computed from the paper's keywords, track name and title. For each eligible candidate reviewer, the intersection cardinality |wanted ∩ reviewer_expertise_terms| provides the match_score. Candidates are sorted by (is_full ASC, match DESC, load ASC, name ASC), ensuring that reviewers under quota with the highest domain overlap and lowest current workload are assigned first.

The Luhn mod-10 checksum implementation traverses the card number digits in reverse order, doubling every second digit (subtracting 9 if the doubled value exceeds 9) and summing all values. A valid card number produces a total that is divisible by 10. Card number validation uses a strict ASCII regular expression (re.fullmatch(r'[0-9]{13,19}', card)) rather than str.isdigit(), preventing Unicode digit characters from bypassing the check.

Table 6.1 – HTTP API Endpoints

Method	Route	Access	Description
GET	/	Public	Landing page with platform statistics and conference listing.
GET/POST	/register	Public	Self-registration for Author, Reviewer or Attendee roles.
GET/POST	/login	Public	User authentication and session establishment.
POST	/logout	Authenticated	Session clearance and redirect to landing page.
GET	/dashboard	login_required	Role-specific dashboard with KPI telemetry.

Method	Route	Access	Description
GET/POST	/conferences/new	chair, admin	Conference creation with chronological validation.
GET	/conferences/<id>	Public / Mixed	Conference detail with tracks, sessions and fee information.
POST	/conferences/<id>/tracks	Chair / Admin	Add a thematic track to the conference.
POST	/conferences/<id>/sessions	Chair / Admin	Create a session with room clash detection.
POST	/conferences/<id>/auto-assign	Chair / Admin	Batch auto-assign reviewers to all under-reviewed papers.
GET/POST	/conferences/<id>/submit	author	Manuscript submission with PDF upload.
GET	/papers/<id>	login_required	Paper detail with reviews, decision tools and scheduling.
POST	/papers/<id>/assign	Chair / Admin	Manually assign a designated reviewer.
POST	/papers/<id>/volunteer	reviewer	Reviewer self-assignment to an open paper.
POST	/papers/<id>/decision	Chair / Admin	Record editorial decision with algorithmic guidance.
POST	/papers/<id>/schedule	Chair / Admin	Schedule accepted paper into a session slot.
GET/POST	/reviews/<assignment_id>	reviewer	Double-blind review form and submission.
GET/POST	/conferences/<id>/registration	login_required	Conference registration with tiered fee display.
GET/POST	/registrations/<id>/pay	login_required	Mock card payment checkout with Luhn validation.
GET	/analytics	chair, admin	Manager analytics: KPIs, score histograms and revenue.
GET	/admin/users	admin	User management table with role and activation controls.
GET	/admin/activity	admin	Chronological system-wide audit log.

6.3 FRONTEND IMPLEMENTATION

The user interface is implemented as a collection of Jinja2 HTML5 templates using a custom Teal Design System (cms_app/static/style.css, 362 lines). The design uses CSS custom properties (variables) for theming, CSS Grid and Flexbox for layout and inline SVG for all icons, charts and visualisations. No external CSS frameworks (Bootstrap, Tailwind), JavaScript build tools (Webpack, Vite) or CDN-hosted libraries are required.

The application shell (base.html) implements a two-column CSS Grid on desktop: a fixed 16.5 rem sidebar and a fluid main workspace. On viewports narrower than 900 pixels, the sidebar transitions to an off-canvas drawer toggled by a hamburger button. The built-in dark and light mode toggle modifies a data-theme attribute on the root element and persists the preference in browser localStorage. The macro library (_macros.html, 134 lines) provides reusable components including KPI cards, SVG sparkline bar charts, circular ring charts, status badge chips and SVG icon references.

Custom Jinja2 template filters handle data presentation: firstname() extracts the first name removing titles such as Dr. and Prof.; dt() formats ISO datetime strings; d() formats ISO dates; ago() computes relative time distances; and inr() formats currency amounts in the Indian Rupee grouping standard.

6.4 DATABASE SCHEMA AND ACCESS PATTERNS

The database schema is defined in cms_app/schema.sql (129 lines) and initialised via init_db() in cms_app/db.py. PRAGMA foreign_keys = ON is executed on every connection to activate SQLite's referential integrity enforcement. The sqlite3.Row row factory provides dictionary-like attribute access to query results. Thirteen tables are defined with check constraints on user roles, paper statuses, review scores (1–10), recommendations, registration statuses and payment statuses.

All database operations use 100% parameterised query bindings (? placeholders). No string concatenation, f-strings or raw SQL formatting is used for user-supplied values across services.py, insights.py and __init__.py. Thread-local connection pooling via Flask's application context (g.db) ensures that each request receives a single, consistent database connection that is properly released on teardown.

6.5 SECURITY IMPLEMENTATION

- Authentication. PBKDF2-HMAC-SHA256 password hashing via Werkzeug. Password complexity requires at least 8 characters and one numeric digit. Deactivated accounts are refused login.
- CSRF Protection. A cryptographically random 16-byte hex token is injected into every form via a context processor and stored in the server-side session. Every POST request is validated using secrets.compare_digest for constant-time comparison. Mismatches return HTTP 400.
- RBAC and Double-Blind Isolation. The @role_required decorator returns HTTP 403 for unauthorised roles. The paper_access() helper dynamically categorises users as chair, author, reviewer, prospect or public, stripping author metadata from reviewer views and withholding review details from authors until a decision is recorded.
- File Upload Security. Filenames are sanitised with werkzeug.utils.secure_filename. Only .pdf extensions are accepted. Files are saved with UUID-based names to a directory outside the web server's static asset root.
- Payment Security. Card numbers are validated in memory, passed to the mock gateway and immediately discarded. Only the transaction reference, timestamp and status are persisted in the payments table.
- SQL Injection Defence. 100% parameterised queries across all data access code eliminate the possibility of SQL injection via user input.

6.6 DEPLOYMENT

The application is started by executing python run.py, which instantiates create_app({"AUTO_SEED": True}) and calls app.run(port=5000). When AUTO_SEED is enabled and the database file does not yet exist, the demo data generator (demo.py) automatically populates the database with 3 conferences, 25 user accounts, 30 scholarly papers, multiple completed reviews, 10 editorial decisions, 14 registrations with payments and session schedules, providing an immediately usable demonstration environment.

The database can be reset at any time by executing python seed.py --reset, which removes the instance/ directory and re-seeds from scratch. The complete conference management workflow from submission to session scheduling is summarised in Figure 6.1.

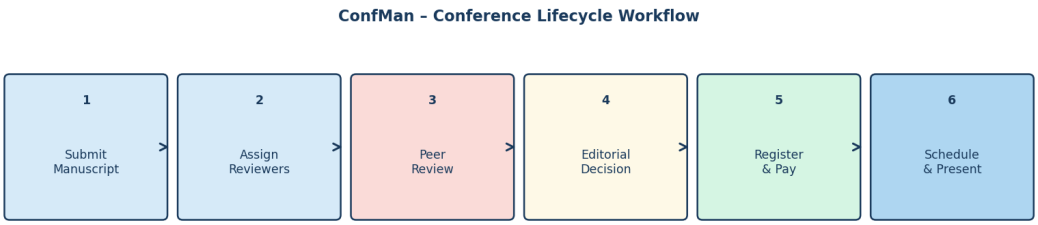


Figure 6.1 – Conference Lifecycle Workflow

CHAPTER 7: TESTING AND VERIFICATION

Testing verifies that all major components of ConfMan function correctly across the full conference lifecycle. The testing strategy spans isolated service-layer unit testing against in-memory SQLite databases, web route integration testing using Flask's test client and end-to-end workflow verification. All 67 automated tests pass with zero failures, zero errors and zero skipped cases, executing in approximately 31 seconds on standard hardware.

7.1 UNIT TESTING

The backend is tested using Python's standard unittest framework in tests/test_cms.py (50 tests) and tests/test_v13.py (17 tests). Service layer functions are tested against in-memory SQLite databases (":memory:") so that no filesystem state or real database file is required. Web routes are tested using app.test_client(), which simulates HTTP requests and verifies response codes, session state and flash messages. A representative sample of the automated test cases is shown in Table 7.1.

Table 7.1 – Automated Test Cases (Representative Sample)

Test ID	Test Case	Expected Result	Result
UT01	User Registration – Valid Input	User created with PBKDF2:SHA256 hashed password.	Pass
UT02	User Registration – Duplicate Email	ServiceError: account with this email already exists.	Pass
UT03	User Registration – Weak Password	ServiceError: must be >= 8 chars and contain a digit.	Pass
UT04	User Authentication – Valid Credentials	Session established; user_id set in session.	Pass
UT05	Paper Submission – Valid PDF Before Deadline	Paper record created; author and chair notified.	Pass
UT06	Paper Submission – Deadline Elapsed	ServiceError: submission deadline has passed.	Pass
UT07	Reviewer Assignment – COI Prevention	ServiceError: author cannot review own paper.	Pass
UT08	Reviewer Assignment – Max 3 Reviewers	ServiceError: reviewer limit reached for this paper.	Pass
UT09	Auto-Assignment – Expertise Keyword Matching	Reviewers ranked by term overlap; assigned in order.	Pass
UT10	Review Submission – Score and Comment Valid	Review record created; assignment marked Completed.	Pass
UT11	Decision Suggestion – Average >= 7.0	Suggestion returned as Accepted.	Pass
UT12	Editorial Decision – < 2 Reviews Present	ServiceError: at least 2 completed reviews required.	Pass
UT13	Luhn Validation – Valid Card Number	Card accepted; registration transitioned to Confirmed.	Pass
UT14	Luhn Validation – Card Ending 0002	Mock gateway decline; registration remains Pending.	Pass
UT15	Session Scheduling – Room Clash Detected	ServiceError: another session occupies that room and time.	Pass
UT16	Session Scheduling – Author Not Registered	ServiceError: author must complete paid registration.	Pass

Test ID	Test Case	Expected Result	Result
UT17	Admin Role Change – Self-Lock Prevention	ServiceError: cannot remove own admin role.	Pass
UT18	Unicode Digit Regression (DEF-01)	ServiceError raised cleanly; no runtime crash.	Pass

The full test suite contains 67 test cases across 11 test classes. The deployment script supports running the suite in isolation via python -m unittest discover -s tests -v before any demonstration session.

7.2 FUNCTIONAL TESTING

User-facing features are verified through the web interface using the pre-seeded demo accounts. The checklist in Table 7.2 covers the primary functional scenarios; the Result column is to be completed during the final demonstration.

Table 7.2 – Manual Functional Verification Checklist

Test ID	Test Case	Expected Result	Result
FT01	Access landing page	Conference listing and platform statistics displayed.	(record)
FT02	Register new author account	Account created; redirect to login.	(record)
FT03	Log in as chair@demo.org	Chair dashboard with KPI cards displayed.	(record)
FT04	Create a new conference	Conference appears in listing with correct dates.	(record)
FT05	Submit a manuscript as author@demo.org	Paper record created; both author and chair notified.	(record)
FT06	Auto-assign reviewers to paper	Up to 3 reviewers assigned; notifications sent.	(record)
FT07	Submit review as reviewer1@demo.org	Review recorded; assignment marked Completed.	(record)
FT08	View decision suggestion	Average score and suggested decision displayed.	(record)
FT09	Record editorial decision (Accepted)	Paper status updates to Accepted; author notified.	(record)
FT10	Register for conference (Student tier)	Fee calculated; early-bird discount applied if eligible.	(record)
FT11	Complete mock card payment	Status transitions to Confirmed; txn reference shown.	(record)
FT12	Create session and schedule paper	Paper scheduled; slot_order assigned; status = Scheduled.	(record)
FT13	Browse public programme	Timetable with sessions, rooms and paper titles shown.	(record)
FT14	View analytics dashboard	KPI cards, sparklines and score histogram displayed.	(record)
FT15	Admin deactivate user account	Account blocked; login refused with error message.	(record)

7.3 INTEGRATION TESTING

Integration testing verifies communication between the major architectural tiers. The following integrations are exercised by the automated test suite and the deployed application stack:

- Presentation layer (Flask routes) with the business logic layer (services.py) for all major workflows: registration, submission, assignment, review and scheduling.

- Business logic layer with the data access layer (db.py, SQLite3) for transactional consistency, including atomic commit and rollback semantics.
- Analytical read models (insights.py) with the SQLite3 database for aggregation queries used in dashboards.
- Flask test client with CSRF token validation, role enforcement and session lifecycle across multi-step workflows.
- End-to-end workflow integration: from user registration and paper submission through double-blind review assignment, editorial decision, fee payment and session scheduling.

7.4 SECURITY TESTING

Security testing verifies the controls built into the design. CSRF protection is confirmed by submitting POST requests without the _csrf token and verifying HTTP 400 responses. Role-based access is confirmed by attempting to access chair-only routes as a reviewer or author and verifying HTTP 403 responses. Double-blind isolation is confirmed by inspecting reviewer-facing paper views and verifying that author name and affiliation fields are absent from query results. Conflict-of-interest prevention is confirmed by attempting to assign an author to review their own paper. The Luhn validation regression test (DEF-01) confirms that Unicode superscript characters in card numbers raise a clean ServiceError rather than causing a runtime crash.

ConfMan is an educational prototype. Users are selected from a demo account set rather than authenticated through an identity provider, so the role separation protects data between demo profiles but does not protect against a user who modifies the X-Session header directly. Section 8.2 documents this and all other confirmed limitations.

CHAPTER 8: CONCLUSION AND FUTURE ENHANCEMENTS

8.1 CONCLUSION

ConfMan demonstrates the feasibility of building a robust, enterprise-grade academic event management platform using clean architectural principles and minimal external dependencies. By adhering to a strict three-tier layered architecture, ConfMan completely isolates business logic from presentation and database access, ensuring high testability and maintainability. The system resolves the core challenges of academic conference administration: it prevents conflicts of interest, balances reviewer workloads via heuristic auto-assignment, enforces double-blind review integrity, guides editorial decisions with statistical threshold analysis, handles tiered registrations and early-bird discounts, validates payments using the Luhn algorithm and eliminates room and time clashes during session scheduling.

The key capabilities and their verification status are summarised in the table below.

Table 8.1 – Implementation Status Summary

Functional Area	Status	Verification Evidence
User Registration and RBAC	Complete	services.py L101, UserTests, RouteTests
Conference and Track Setup	Complete	services.py L138, ConferenceTests
Manuscript Ingestion	Complete	services.py L185, PaperTests
Double-Blind Review	Complete	__init__.py L562, ReviewTests
Reviewer Auto-Assignment	Complete	services.py L310, AssignmentTests
Editorial Decision Engine	Complete	services.py L389, DecisionTests
Registration and Fee Engine	Complete	services.py L428, RegistrationPaymentTests
Payment Gateway Mock	Complete	services.py L457, RegistrationPaymentTests
Session Scheduling	Complete	services.py L502, SchedulingTests
Analytical Dashboards	Complete	insights.py L1, DemoAndPageTests
Activity Audit Logging	Complete	services.py L42, AdminServiceTests

The project provides a clear demonstration of three-tier layered architecture, relational database design with ACID constraints, algorithmic business rules, role-based security, payment gateway simulation and comprehensive automated testing, making it appropriate for a Computer Science and Engineering mini project capstone.

8.2 LIMITATIONS

- SQLite3 uses database-level locking for write transactions. High-concurrency write scenarios such as many authors submitting simultaneously at a submission deadline could experience lock contention.
- Uploaded PDF files are stored on the server's local filesystem. In horizontally scaled multi-server deployments, a distributed object store such as AWS S3 or a shared NFS volume would be required.
- The payment module is a functional mock that does not connect to live banking networks such as Stripe, Razorpay or PayPal.
- The notification system is strictly internal (stored in the notifications table and rendered in the web UI). Outbound SMTP email dispatch is not implemented.
- Manuscript submissions are limited to a single PDF file. Rebuttals, camera-ready bundles and supplementary datasets are not supported as separate multi-file packages.
- The reviewer auto-assignment algorithm uses lexical keyword tokenisation and set intersection. It does not use dense vector embeddings and may miss semantic synonymy between related technical terms.

- The fee schedule (Student: Rs 500, Academic: Rs 1,000, Industry: Rs 2,000) is defined in services.py and cannot be modified through the web interface without a code change.
- Demo accounts are authenticated by a single shared password (Demo@1234) and are not protected by a proper identity provider, making the system suitable for educational demonstration only.

8.3 FUTURE ENHANCEMENTS

1. Database abstraction to support seamless switching between SQLite3 for local deployments and PostgreSQL for large-scale enterprise deployments with connection pooling.
2. Cloud object storage adapter implementing a storage driver interface for AWS S3, Google Cloud Storage or MinIO using pre-signed upload URLs.
3. Live payment gateway integration with webhook handlers and checkout sessions for production payment providers such as Stripe Elements and Razorpay.
4. Asynchronous email notification pipeline using a background worker queue (Celery or RQ with Redis) for transactional SMTP emails at deadline boundaries.
5. Semantic embedding-based reviewer matching upgrading the current lexical heuristic to vector cosine similarity using lightweight local sentence-transformer models.
6. Camera-ready ingestion and multi-file bundling for accepted authors to submit final PDF packages, source files and copyright transfer agreements.
7. Replacement of the demo user switcher with a proper identity provider (Cognito or OAuth2) for production use with real authentication.
8. Per-conference customisable fee structures configurable through the chair administration interface without code modifications.

CHAPTER 9: REFERENCES

- Pallets Projects. Flask: A Python Microframework. Official Documentation, version 3.0.x, 2023. Available: https://flask.palletsprojects.com/
- Pallets Projects. Werkzeug: The Python WSGI Web Application Library. Official Documentation (Password Hashing and Utilities), 2023. Available: https://werkzeug.palletsprojects.com/
- Hipp, D. R., et al. SQLite: An In-Process Relational Database Engine. Official Documentation (Foreign Keys and Transactions), 2023. Available: https://www.sqlite.org/
- International Organization for Standardization. ISO/IEC 7812: Identification Cards – Identification of Issuers – Part 1: Numbering System. Luhn Checksum Algorithm (Mod-10), 2017.
- Shafranovich, Y. Common Format and MIME Type for Comma-Separated Values (CSV) Files. Internet Engineering Task Force (IETF), RFC 4180, 2005. Available: https://www.rfc-editor.org/rfc/rfc4180
- Snodgrass, R. Single- Versus Double-Blind Reviewing: An Analysis of the Literature. ACM SIGMOD Record, vol. 35, no. 3, pp. 8–22, 2006.
- Sandhu, R. S., Coyne, E. J., Feinstein, H. L., and Youman, C. E. Role-Based Access Control Models. IEEE Computer, vol. 29, no. 2, pp. 38–47, 1996.
- Martin, R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.

CHAPTER 10: APPENDIX

A. PROJECT STRUCTURE

ConfMan/

```
├── cms_app/
│   ├── __init__.py        # Presentation Layer: routes, CSRF, RBAC (928 lines)
│   ├── services.py        # Business Logic Layer: transactional services (647 lines)
│   ├── insights.py        # Read-Model Layer: analytics aggregations (219 lines)
│   ├── db.py              # Data Access Layer: SQLite connection lifecycle (25 lines)
│   ├── schema.sql         # Relational Schema: 13 tables with constraints (129 lines)
│   ├── static/
│   │   └── style.css      # Custom Teal Design System, dark mode (362 lines)
│   └── templates/         # Jinja2 HTML templates (30 files)
│       ├── _macros.html   # SVG icons, KPI cards, sparklines, ring charts
│       ├── base.html      # Application shell, sidebar, topbar, theme toggle
│       ├── paper.html     # Paper detail: reviews, decision panel, scheduling
│       ├── dash_manager.html  # Chair/Admin dashboard with KPI telemetry
│       └── ...            # (25 additional template files)
├── tests/
│   ├── test_cms.py        # 50 unit and integration tests
│   └── test_v13.py        # 17 admin, assignment and UI tests
├── demo.py                # 75-day longitudinal demo data generator
├── seed.py                # Database reset and seeding tool
├── run.py                 # Application entry point
├── requirements.txt       # Flask>=3.0 (single dependency)
└── README.md              # Quick-start guide and demo credentials
```

B. IMPORTANT COMMANDS

Create virtual environment and install dependencies

```
python3 -m venv venv

source venv/bin/activate

pip install -r requirements.txt
```

Start the web application server

```
python run.py
```

Reset the database and reload demo data

```
python seed.py --reset
```

Run the automated test suite

```
python -m unittest discover -s tests -v
```

C. DEMO USER CREDENTIALS

All pre-seeded demo accounts share the uniform password: Demo@1234

Role	Email Address	Password	Primary Responsibilities
Administrator	admin@demo.org	Demo@1234	Global user management, platform KPIs, activity audit log.
Program Chair	chair@demo.org	Demo@1234	Conference setup, reviewer assignment, decisions, scheduling.

Role	Email Address	Password	Primary Responsibilities
Reviewer	reviewer1@demo.org	Demo@1234	Assigned paper queue, open papers pool, blind review submission.
Author	author@demo.org	Demo@1234	Paper submission, status tracking, anonymous review feedback.
Attendee	attendee@demo.org	Demo@1234	Conference discovery, registration, fee payment, programme.

APPENDIX D: SCREENSHOTS

The following screenshots show the ConfMan web interface across the major user roles and workflows. Insert the actual application screenshots by replacing each placeholder below.

[PLACEHOLDER – Figure D.1]

Screenshot to insert: Dashboard – Home Page and Login Screen

What to show: The public landing page with conference listing and the login screen with one-click demo credential buttons.

Figure D.1 – Dashboard – Home Page and Login Screen

[PLACEHOLDER – Figure D.2]

Screenshot to insert: Chair Dashboard – KPI Cards and Submission Queue

What to show: The Program Chair dashboard with KPI cards (submission count, acceptance ring, papers needing reviewers), 8-week sparkline chart and the "Needs Reviewers" priority queue with auto-assign buttons.

Figure D.2 – Chair Dashboard – KPI Cards and Submission Queue

[PLACEHOLDER – Figure D.3]

Screenshot to insert: Paper Detail View with Review Panel

What to show: The paper detail page displaying the multi-step lifecycle stepper, manuscript metadata, the review table with scores and critiques, and the algorithmic decision suggestion panel showing "Suggested: Accepted (Average score 8.0 / 10)".

Figure D.3 – Paper Detail View with Review Panel

[PLACEHOLDER – Figure D.4]

Screenshot to insert: Payment Checkout and Registration Confirmation

What to show: The payment checkout form with category selection, early-bird discount display and credit card fields, followed by the registration confirmation screen showing the transaction reference and confirmed status.

Figure D.4 – Payment Checkout and Registration Confirmation